

INTERNSHIP REPORT

Design and Development of the PayMedia Inclusive Monetization Platform

Student: Nathan

Academic Program: DUT (Diplôme Universitaire de Technologie)

Role: Software Development Intern

Environment: macOS, Flutter, Python (FastAPI), PostgreSQL, Redis

Date: June 2026

1. Introduction & Context

This report documents the architectural design, core functional engineering, and development strategies executed during my software development internship. The assigned mission focused on **PayMedia**, an innovative fintech-driven ecosystem aimed at bridging the gap between traditional broadcast/print media audiences and digital micro-transactions.

Traditional media channels (TV, Radio, Press) struggle with direct digital conversion due to onboarding friction and payment processing inefficiencies. The pedagogical objective of this project within my DUT curriculum was to build an absolute "Zero Friction" micro-payment highway that guarantees universal access across distinct regional field conditions.

2. Ecosystem & Requirements Engineering

The core business logic coordinates transactions across four macro-actors. Part of my objective was translating business needs from the Master SFD into programmatic constraints:

Actor	System Role	Implemented Feature Matrix
End User	Consumes paid items via 1-Click execution	Biometric validation, Offline SQLite vault, Vocal text-to-speech engine
Media Partner	Ingests original media and maps triggers	Drag-and-Drop file processing dashboard, real-time analytics stream
Payment Gateway	Processes high-concurrency wallet recharges	Asynchronous webhooks for Orange, MTN, Wave, and standard credit networks

3. Applied Architectural & Engineering Specifications

3.1 Core Tech Stack Selection

Developing on a primary macOS environment, the architecture was selected to maintain structural speed, strict ACID transactional compliance, and cross-platform flexibility:

- **Frontend Application:** Built using **Flutter (Dart)**. This framework provided a clean, hardware-optimized codebase to handle low-level device components, such as camera matrices for rapid QR acquisition and microphone streams for acoustic parsing.
- **Backend Layer:** Powered by **Python (FastAPI)**. Chosen for its native asynchronous orchestration capabilities, matching response speed targets of under 800ms during simulated media broadcast spikes.
- **Database Engine:** Managed through a dual layer: **PostgreSQL** for uncompromised financial ledger tracking, and **Redis** for volatile short-code in-memory lookup caching.

3.2 Mitigating Real-World Constraints

The Micro-Payment Profitability Challenge:

Direct operator transaction processing for small item sales fails due to heavy fixed gateway processing premiums (\$f\$). To resolve this, I implemented a tiered wallet allocation architecture:

Net Earnings = $P - [f + (v \times P)]$ is bypassed for single transactions by introducing pre-funded internal balances where $S_n = S_{n-1} - P_{item}$ operates at zero fixed transaction friction.

4. Implemented Technical Modules

4.1 M-01: Onboarding & Multi-Channel Acquisition

The ingestion module supports three decoupled capture interfaces to optimize accessible user loops: Image analysis mode for physical QR press prints, manual numeric Short-Code arrays (e.g., #302) running structural $O(1)$ database query lookups for low-tier

hardware, and an acoustic watermarking module that processes 4-second audio samples to extract media IDs.

4.2 M-02: Secure Readers & Advanced Accessibility

To support areas experiencing severe digital network drops, an offline local encryption stack using **AES-256** was developed within the mobile client's persistent cache. Additionally, network-aware monitors scale media downward automatically—switching heavy layouts to lightweight text-only variants when data pipelines drop below 100 kbps. A custom text-to-speech accessibility module was built to assist visually impaired individuals.

| 5. Project Outcomes & Conclusion

This internship provided deep practical exposure to high-concurrency API modeling, algorithmic optimization for financial engines, and mobile-native component programming. The resulting framework successfully adheres to the 99.9% availability criteria outlines. The project stands validated and structured for production implementation.